

AI WEATHERWISE

Phase 2: Requirement Analysis

Functional, non-functional, data and system requirements

Project	AI WeatherWise – Deliver intelligent weather forecasts and insights
Phase	2 of 8 – Requirement Analysis
Technology	Node.js, Express.js, MongoDB (Mongoose), JWT, Google Gemini AI, OpenWeatherMap
Version / Date	1.0 28 September 2026

1. Purpose and Scope

This document lists what the AI WeatherWise backend must do and the conditions under which it must run. It is derived from the ideation phase and the project reference document, and it is the basis for design, development and testing.

In scope: user authentication, favorite location management, real-time weather retrieval, AI-driven summaries and recommendations, security and error handling. **Out of scope (first release):** graphical front end, alerts, subscriptions and weather history.

2. Stakeholders and Roles

Role	Responsibilities
Admin	User account management (create, delete, suspend); oversight of API health and system logs; monitoring and managing database integrity.
Registered User	Creates and maintains a list of favorite locations; requests real-time weather; interacts with Gemini AI for summaries and activity recommendations; manages own credentials and profile.
Weather Application User	Consumes the public weather endpoints without an account.
Developer	Builds, tests and verifies weather and AI integrations.

3. Functional Requirements

ID	Requirement	Priority
FR-01	The system shall allow a new user to register with name, email and password.	Must
FR-02	The system shall authenticate users by email and password and return a signed JWT.	Must
FR-03	Passwords shall be stored only as bcrypt hashes.	Must
FR-04	A registered user shall be able to add a favorite location (city and country).	Must

ID	Requirement	Priority
FR-05	A registered user shall be able to view, update and delete their own favorite locations.	Must
FR-06	The system shall prevent the same city and country from being saved twice by the same user.	Must
FR-07	The system shall return current weather (temperature, humidity, wind speed, condition) for a requested city.	Must
FR-08	The system shall generate an AI recommendation (clothing and activities) from the given weather data.	Must
FR-09	The system shall generate a natural-language AI weather summary.	Should
FR-10	The system shall run in fallback (mock) mode when weather or AI API keys are missing, marking the response accordingly.	Must
FR-11	The system shall validate the JWT on protected routes and reject invalid or missing tokens.	Must
FR-12	A registered user shall be able to update their profile details.	Should
FR-13	An admin shall be able to manage user accounts and monitor API health.	Could
FR-14	The system shall return errors in a consistent JSON structure through centralized error handling.	Should

4. Non-Functional Requirements

Category	Requirement
Security	JWT-based authentication; bcrypt hashing; role-based route guarding; sanitization against XSS and NoSQL injection; secrets kept in the <code>.env</code> file.
Performance	Typical API responses within one second under local conditions (observed 90–475 ms during testing).
Reliability	Fallback mode for external services; server starts only after a successful database connection.
Maintainability	MVC and modular structure: routes, controllers, middleware, models, services, config.
Portability	Runs on Windows 10/11, macOS or Linux with Node.js.
Usability	Consistent JSON responses using a <code>success</code> flag with <code>data</code> or <code>message</code> .
Data integrity	Mongoose schemas with required fields, trimming, unique email and unique user–city–country index.

5. User Stories

ID	As a...	I want to...	So that...
US-01	Traveller	register and log in securely	my saved cities are protected.
US-02	Traveller	save favorite cities	I can track them quickly.
US-03	Traveller	see current weather of a city	I know the conditions before I go.

ID	As a...	I want to...	So that...
US-04	Traveller	get AI advice on clothes and activities	I do not have to interpret raw numbers.
US-05	Public user	read weather without an account	I can use the service instantly.
US-06	Admin	monitor users and API health	the platform stays reliable.

6. System Requirements

6.1 Software Requirements

Component	Requirement	Purpose
Operating System	Windows 10/11, macOS or Linux	Cross-platform operation
Node.js	v16 or above	Runtime for server logic and routing
npm	v8 or above	Dependency management
Gemini AI SDK	@google/generative-ai (Gemini SDK)	Natural-language insights
MongoDB	Local or cloud instance	Stores users and locations
Postman	Latest version	API verification and testing
Code Editor	Visual Studio Code or similar	Development environment

6.2 Hardware Requirements

Component	Requirement
Processor	Intel Core i5 (8th Gen or above) / AMD Ryzen 5 or equivalent
RAM	8 GB minimum (16 GB recommended for concurrent MongoDB, processing layers and execution nodes)
Storage	1 GB of available disk space

7. Data Requirements

User

Field	Type	Rules
_id	ObjectId	Primary key
name	String	Required, trimmed
email	String	Required, unique, lowercase, valid e-mail pattern
password	String	Required, minimum 6 characters, stored hashed
role	String	Default "reader"
createdAt / updatedAt	Date	Automatic timestamps

Location

Field	Type	Rules
_id	ObjectId	Primary key
user	ObjectId (ref User)	Required – owner of the favorite
city	String	Required, trimmed
country	String	Required, trimmed
createdAt	Date	Default: now (timestamps enabled)
Index	{ user, city, country }	Unique – no duplicate favorites per user

8. External Interface Requirements

Interface	Description
OpenWeatherMap API	Provides current weather metrics for a city.
Google Gemini AI API	Generates weather summaries and personalized recommendations from live temperature and conditions.
MongoDB	Persistent storage for the User and Location collections through Mongoose.
Client (Postman / any HTTP client)	Sends JSON requests (GET, POST, PUT, DELETE) and receives JSON responses.

9. Environment Configuration Requirements

Variable	Purpose
PORT	Port on which the Express server listens (default 5000)
MONGO_URI	MongoDB connection string
JWT_SECRET	Secret used to sign and verify JWT tokens
GEMINI_API_KEY	API key for Google Gemini (optional – fallback mode is used if absent)

10. Assumptions, Constraints and Acceptance Criteria

- **Assumption:** MongoDB is running and reachable before the server starts.
- **Constraint:** external weather and AI services depend on network access and valid API keys.
- **Constraint:** first release is API-only; there is no graphical user interface.
- **Acceptance:** every "Must" requirement (FR-01 to FR-08, FR-10, FR-11) is demonstrated through Postman tests with successful responses.

The next phase, **Project Design**, describes the architecture, data model and API structure that satisfy these requirements.